Designed by Remus Teodorescu 2026

THE ARC

From Arrays to Measurements

WHAT CHANGES WHEN THE NUMBERS COME FROM AN INSTRUMENT

Last time an array was a mathematical object. Today it carries measurements, and three things change. The numbers have units. Some of them are missing. And there is a physical model that says what they should have been.

Pandas exists for the first two. It is NumPy with labels on the axes and a coherent story about missing values, and it is what every measurement file in engineering practice arrives as.

SciPy exists for the third. Its integrator solves the differential equation your physics gives you, and its optimiser fits that solution's constants to your data.

The argument underneath all of it is on the right, and it is the reason this half-lecture comes before any neural network. A two-parameter fit of a physically motivated form is a strong baseline. Knowing how strong is the only way to tell later whether a network was worth the trouble.

TODAY, IN FOUR PARTS

pandas	tables, and selecting from them
file I/O	a CSV that fights back
solve_ivp	an ODE, solved numerically
curve_fit	two parameters, with uncertainty

No new mathematics. Every equation today is one you have seen in a thermodynamics course.

THE ARGUMENT OF THIS LECTURE

Deep learning has to beat something. Today you build the something, so that in three weeks the comparison is real rather than rhetorical.

A DataFrame Is an Array with Labels

AND ONE COLUMN AT A TIME, NOT ONE ROW AT A TIME

A DataFrame is a two-dimensional table with a name on every column and an index on every row. Each column is a NumPy array with a dtype of its own, so one table can hold floats, integers, timestamps and text side by side — which is what a logger file actually contains.

Think of it as a dictionary of columns rather than a list of rows. That is how it is stored and it is how you should operate on it: whole columns at a time, exactly as you learned to operate on whole arrays last lecture.

The index is the part with no NumPy equivalent. It labels the rows, it need not be integers, and when it is a time it unlocks resampling and interpolation for free.

Everything from L1.2 still applies underneath. A column is an array; broadcasting works; the shape rule is unchanged. Pandas adds labels and missing-value handling, and takes nothing away.

THE OBJECT

index	time_s	temp_C	status
0	0.0	80.05	OK
1	2.0	78.11	OK
2	4.0	NaN	FAULT
3	6.0	74.90	OK

THREE THINGS TO CALL FIRST

```
df.head()      the first five rows
df.dtypes      one dtype per column
df.describe()  count, mean, min, max
# and df.shape, every time
```

read_csv, and Its Six Useful Arguments

ONE FUNCTION DOES ALMOST ALL OF IT

read_csv reads a delimited text file into a DataFrame. In the easy case that is the whole story and you never think about it again. In the real case you will need a handful of its arguments, and it is worth knowing which six before you meet a file that needs them.

sep, when the file is not comma-separated — European exports are usually semicolons. skiprows, for the header block a logger writes. decimal, for the comma decimal separator used across most of continental Europe, which is the single most common reason a temperature column arrives as text.

na_values, for whatever the instrument writes when a channel drops out. parse_dates, for timestamp columns. And usecols, when the file has forty channels and you want three.

The habit that saves you: after every read, look at dtypes. A numeric column that came in as object is a file problem, and finding it now costs a second rather than finding it later inside a fit.

THE ARGUMENTS THAT MATTER

```
import pandas as pd

df = pd.read_csv(
    "cooling_log.csv",
    sep=";",           # not a comma
    decimal=",",       # European decimals
    skiprows=2,        # logger preamble
    na_values=["", "SENSOR FAULT"],
    usecols=["time", "temp_C"],
)

df.dtypes            # look, every time
```

An object dtype where you expected a float is a file problem, not a pandas problem.

Selecting: loc, iloc, and Masks

TWO INDEXERS, AND KNOWING WHICH ONE YOU ARE USING

Square brackets with a column name give you that column, as a Series — a one-dimensional labelled array. Square brackets with a list of names give you a smaller DataFrame. That covers most days.

For anything else there are two indexers and the difference matters. loc selects by label, and — unlike everything in L1.2 — its slices include their endpoint, because a label range is inclusive. iloc selects by integer position and behaves exactly like NumPy, endpoint excluded.

Mixing them up gives you an off-by-one that does not raise, so say which you meant. The rule I use: iloc when I am thinking about positions, loc when I am thinking about times or names.

Boolean masks work as they did last lecture and are the right way to express a condition on the data. The temperatures above fifty degrees; the rows where the status is OK. Same idea, same ampersand, same requirement for brackets around each condition.

THE FORMS YOU WILL USE

```
df["temp_C"]              a Series
df[["time_s", "temp_C"]]  a DataFrame

df.iloc[0:5]              by position, [0,5)
df.loc[0:5]               by label,    [0,5]

df.loc[df["temp_C"] > 50.0]
df.loc[df["status"] == "OK", "temp_C"]

t = df["time_s"].to_numpy()
T = df["temp_C"].to_numpy()
```

The last two lines are the exit door. Everything after slide ten is NumPy.

What a Logger Actually Gives You

NINE LINES, AND SIX SEPARATE PROBLEMS

Every tutorial dataset is clean. No dataset you are given at work is clean, and the gap between those two facts is where a surprising amount of engineering time goes.

On the right is a file of the kind you will actually be handed. It is short enough to read in full and it contains six distinct problems, none of which is unusual and all of which will make read_csv give you something wrong rather than something that fails.

Read it before I list them. Look at the separator, look at the numbers, look at line six, and look at what the fourth column is doing.

The general lesson is worth stating before the specifics: a file that loads without an error has not necessarily loaded correctly. Checking dtypes and calling describe takes ten seconds and is the whole defence.

cooling_log.csv, UNEDITED

```
# Datalogger v2.3  exported 2026-02-11 09:14
# channel 1: block thermocouple  channel 2: ambient
time;temp_C;ambient;status
0,0;80,05;22,0;OK
2,0;78,11;22,0;OK
4,0;;22,0;SENSOR FAULT
6,0;74,90;22,0;OK
8,0;73,52;22,1;OK
10,0;72,08;22,0;OK
```

Two comment lines, a header, six rows of data. Everything wrong with it is visible on this slide.

PAUSE THE VIDEO HERE

Find as many problems as you can before the next slide. Six is the number I have counted; there is arguably a seventh.

Six Problems, Six Arguments

FIVE LOAD WITHOUT AN ERROR. THE PREAMBLE RAISES ONE, AND BLAMES THE WRONG LINE

SEMICOLONS

The separator is a semicolon, so read_csv puts the entire row into one column. sep=";"

DECIMAL COMMAS

80,05 is eighty point nought five. Without decimal="," the column arrives as text and every later arithmetic fails.

A PREAMBLE

Two comment lines above the header, and the only one of the six that raises: pandas takes one field from line one and finds four on line four. skiprows=2, or comment="#".

AN EMPTY FIELD

Line six has no temperature. It becomes NaN, which is correct — and then propagates through every sum you take.

A SENTINEL IN A COLUMN

SENSOR FAULT is text in a status column, and it marks a row whose reading you must not use. na_values, then drop.

NO UNITS ANYWHERE

temp_C is a hint, not a guarantee, and ambient has no unit at all. The only fix is asking whoever produced the file.

BEFORE: sep AND comment ONLY

time	temp_C	ambient	status
0,0	80,05	22,0	OK
2,0	78,11	22,0	OK
4,0	NaN	22,0	SENSOR FAULT
6,0	74,90	22,0	OK
8,0	73,52	22,1	OK
10,0	72,08	22,0	OK

dtypes: object, object, object, object. It loaded, it looks like data, and no arithmetic on it will work.

AND THE ARGUABLE SEVENTH

The ambient column is constant to one decimal place for every row, which usually means it is a setting rather than a measurement. That is not a file format problem and no argument to read_csv will fix it — but it changes what you are allowed to conclude, and it is the kind of thing only reading the file will tell you.

Missing Data: Three Honest Choices

AND ONE DISHONEST ONE

A missing value in pandas is NaN, and NaN propagates: anything arithmetic you do with it is NaN. That is a feature, because it means a missing value cannot quietly become a number.

You have three defensible responses, on the right, and which you choose depends on why the value is missing rather than on convenience. Dropping the row is right when the reading is absent. Interpolating is right when the quantity is smooth and the gap is short. Keeping the NaN and using functions that skip it is right when you want the count of what you actually measured.

The dishonest option is filling with zero. A missing temperature is not zero degrees, and a zero will drag every mean, every fit and every plot towards itself while looking like data. If you ever see fillna of nought in somebody's analysis, ask about it.

Whatever you choose, say so in the report, and say how many rows it affected. A fit to twenty-nine of thirty-one samples is a different claim from a fit to thirty-one.

THE THREE

dropna()	the reading is absent — drop the row
interpolate()	smooth quantity, short gap
keep the NaN	use mean(), which skips it

AFTER: SIX ARGUMENTS, THEN dropna

time	temp_C	ambient	status
0.0	80.05	22.0	OK
2.0	78.11	22.0	OK
6.0	74.90	22.0	OK
8.0	73.52	22.1	OK
10.0	72.08	22.0	OK

THE ONE TO REFUSE

fillna(0.0). A missing temperature is not zero degrees.

Aggregating, Briefly

TWO OPERATIONS THAT WOULD OTHERWISE BE LOOPS

groupby splits a table by the values in a column, applies a reduction to each group, and puts the results back together. One line for what would otherwise be a loop, a dictionary and an accumulation.

In an engineering context the grouping column is usually a run identifier, an operating point, or a sensor name. Six gauges, mean and standard deviation each: one line.

resample is the same idea on a time index. It is how you turn a logger's irregular ten-hertz stream into a regular one-second series, and it is worth knowing that this is an interpolation and therefore a modelling decision rather than a formatting one.

That is all I will say about pandas. It is a large library and you will need perhaps five per cent of it. The rest is worth looking up on the day you need it rather than learning now.

ONE LINE EACH

```
# mean and spread per gauge
g = df.groupby("gauge")["strain"]
g.agg(["mean", "std", "count"])

# a regular 1 s series from an
# irregular log
reg = df.set_index("t").resample("1s").mean()

# resampling interpolates. That is a
# modelling choice, not formatting.
```

AND THAT IS ENOUGH PANDAS

You will need about five per cent of this library. Look the rest up on the day, and do not read the documentation cover to cover.

And Then Leave

THE BOUNDARY, AND WHY IT IS A GOOD ONE

Pandas is for getting data in and cleaning it. Once it is clean, leave. Call to_numpy on the columns you need and do the rest of your work in arrays.

The reason is that everything downstream — SciPy, PyTorch, every model in this course — takes arrays, and each round trip through pandas is another place for an index to be silently realigned. Pandas aligns on the index when it combines objects, which is a genuinely useful feature and a surprising one if you were expecting positional behaviour.

Check two things as you cross the boundary. The shape, because that is the habit from last lecture. And the dtype, because a column that came in as text will still be text here, and it will not become a float by being passed to something that wants one.

Structurally, a clean notebook has a pandas section at the top, a to_numpy line, and no pandas after it. If pandas appears three-quarters of the way down, something is being redone.

THE EXIT, AND ITS TWO CHECKS

```
clean = df.dropna(subset=["temp_C"])

t = clean["time_s"].to_numpy()
T = clean["temp_C"].to_numpy()

print(t.shape, T.shape)   (29,) (29,)
print(t.dtype, T.dtype)   float64 float64
```

One direction only. Nothing goes back.

Why Solve It Numerically at All

BECAUSE ALMOST NOTHING HAS A CLOSED-FORM SOLUTION

The block cooling in a room gives you an energy balance: the rate of change of stored energy equals the heat lost by convection at the surface. That is one first-order differential equation, and it is separable, so it has an exact solution.

That is the exception. Make the convection coefficient depend on temperature, add radiation, allow the room to warm up, or couple two blocks together, and the closed form disappears while the equation stays perfectly well behaved.

So the honest default is to solve numerically and to keep the analytic case as a test. That is what slide fourteen does, and it is a habit worth building now rather than at the point where you have nothing to check against.

There is a further reason this comes before any neural network. In lecture seven a network is trained by making its residual small — the residual of an equation of exactly this kind. Solving one by conventional means first means you know what a solution looks like before you meet a stranger method for producing one.

$$m\,c_p\,\frac{dT}{dt} = -h\,A(T - T_\infty)$$

THE ONE CASE WITH AN EXACT ANSWER

$$T(t) = T_\infty + (T_0 - T_\infty)e^{-t/\tau}, \qquad \tau = \frac{m\,c_p}{h\,A}$$

Separable, integrable, and the reason this example was chosen: you can check the numerical answer against it.

AND FOUR CASES WITHOUT ONE

h = h(T)	convection varies with ΔT
+ radiation	a fourth power appears
T∞ = T∞(t)	the room warms up too
two blocks	a coupled system

Every Solver Wants the Same Thing

A FIRST-ORDER SYSTEM, AND A FUNCTION THAT RETURNS ITS RATES

Numerical integrators accept exactly one form: a first-order system. You give a state vector, an initial condition, and a function that takes the current time and state and returns the rate of change of each component.

That is the equation at the top right, and it is worth recognising because it is the interface for every integrator in every language you will use.

A higher-order equation is reduced to that form by making the derivatives part of the state. A second-order equation becomes a two-component system: position and velocity, whose rates are velocity and acceleration. That is a mechanical transformation, not a clever one, and you will use it in lecture eleven for a vehicle and in lecture twelve for a swing equation.

So the whole of the modelling work is writing the right-hand side function. It takes t and y, it returns the derivatives, and it must accept them in that order even when it does not use t.

THE ONLY FORM A SOLVER ACCEPTS

$$\frac{d\mathbf{y}}{dt} = \mathbf{f}(t, \mathbf{y}), \qquad \mathbf{y}(t_0) = \mathbf{y}_0$$

$$\ddot{x} = g(t, x, \dot{x}) \implies \mathbf{y} = (x, \dot{x}),\ \dot{\mathbf{y}} = (\dot{x},\ g)$$

THE RIGHT-HAND SIDE, IN FULL

```
def cooling_rhs(t, y, tau, T_inf):
    """dT/dt in K/s. y = (T,)."""
    T, = y
    return [-(T - T_inf) / tau]

# t comes first, even when unused
```

solve_ivp, and What to Set

FOUR ARGUMENTS THAT MATTER, AND TWO THAT USUALLY DO NOT

The call takes the right-hand side, the time span as a pair, the initial state as a sequence, and — the argument people forget — t_eval, the times you actually want output at. Without it you get whatever times the adaptive stepper happened to visit, which is correct but awkward to plot against data.

The method defaults to RK45, an adaptive explicit Runge–Kutta scheme, and that is the right first choice for almost everything you will meet in this course. If it becomes very slow and takes tiny steps, your system is probably stiff, and the answer is to switch to an implicit method rather than to wait.

rtol and atol control the error the stepper is willing to accept. The defaults are loose — a relative tolerance of one in a thousand — and tightening them is the first thing to try when a solution looks slightly wrong rather than badly wrong.

The result object holds t and y. Note that y comes back with shape number of states by number of times, so a single-state system gives you shape one by N and you want the first row. That is a length-one axis, and it is exactly the L1.2 slide eleven trap waiting to happen.

THE CALL

```
from scipy.integrate import solve_ivp

sol = solve_ivp(
    cooling_rhs,
    t_span=(0.0, 60.0),
    y0=[80.05],
    t_eval=np.linspace(0.0, 60.0, 301),
    args=(60.0, 22.0),
    method="RK45", rtol=1e-8, atol=1e-10,
)

sol.y.shape          (1, 301)
T_num = sol.y[0]     (301,)
```

The (1, 301) is a length-one axis. You met that trap last lecture.

Never Believe a Solver Unchecked

THREE CHECKS, IN DECREASING ORDER OF AVAILABILITY

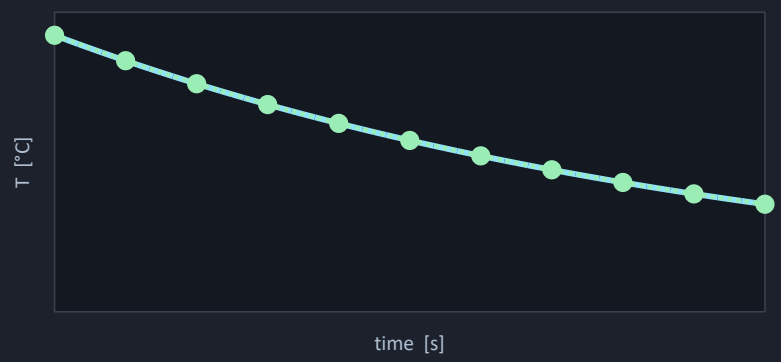
First, and best: compare against an analytic solution on a case that has one. Here it does, so solve the cooling equation numerically, evaluate the exponential, and subtract. With the tolerances on the previous slide the difference should be at the level of one part in ten to the eight, and if it is not, something in your right-hand side is wrong.

Second: check a conserved quantity. Most engineering systems conserve something — energy, mass, charge — and a solver that has drifted will show it there first. This works even when no analytic solution exists, which is most of the time.

Third, and weakest but always available: halve the tolerance and see whether the answer moves. If tightening rtol by two orders of magnitude changes your result in the third significant figure, the third significant figure was never yours to quote.

This is not pedantry. In lecture eight onwards you will validate physics-informed networks with exactly these three checks, in exactly this order, because there is often no test data to validate against at all.

NUMERICAL AGAINST ANALYTIC



Line: exponential. Dots: RK45. Max difference ≈ 1e-8 °C.

THE THREE, RANKED

best	against an analytic case
usually	a conserved quantity
always	halve the tolerance, look again

Fitting a Model with Two Knobs

THE SAME THREE INGREDIENTS AS EVERY MODEL IN THIS COURSE

You have the functional form from the physics: an exponential decay towards ambient, with a time constant. What you do not have is the time constant, because it depends on a convection coefficient that a handbook gives to about thirty per cent.

So estimate it from the data. Choose the parameters that minimise the sum of squared differences between the model and the measurements — the equation below, which is the same least-squares objective you will meet again in lecture four point one with a neural network in place of the exponential.

That substitution is the whole of the conceptual difference. A parameterised function, a loss, and something that minimises it. Today the function has two parameters and the minimiser is Levenberg–Marquardt; in lecture six it will have two thousand and the minimiser will be Adam.

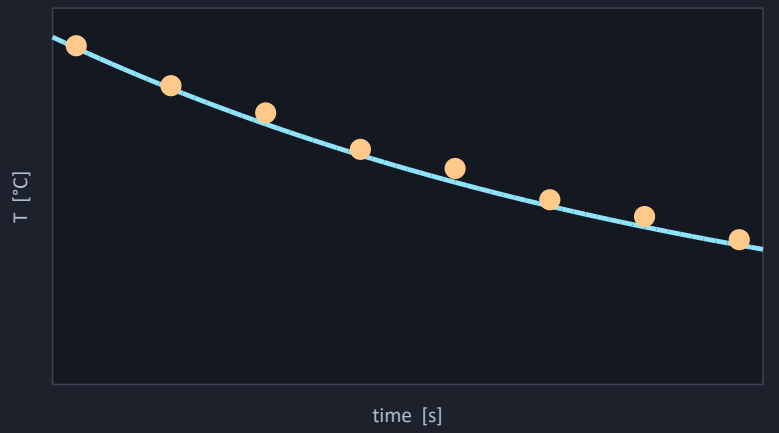
It is worth being explicit that this is not a lesser method. A fitted physical form is the middle of the modelling spectrum, and it is what almost every correlation in your handbook actually is.

$$\hat{\theta} = \operatorname{argmin}_{\theta} \sum_{k=1}^{N} \left(f(t_k, \theta) - T_k \right)^2$$

THE MODEL FUNCTION

TWO FREE PARAMETERS

```
def cooling_model(t, T0, tau):
    """T in °C. t in s. T_inf fixed."""
    return 22.0 + (T0 - 22.0) * np.exp(-t / tau)
```



Two numbers, chosen to make the vertical distances as small as possible.

curve_fit, and the Second Return Value

THE ARGUMENT PEOPLE SKIP, AND THE OUTPUT THEY IGNORE

curve_fit takes the model function, the inputs, the measurements, and an initial guess. It returns two things: the fitted parameters, and their covariance matrix.

The initial guess is not optional in practice, even though the argument is. Least squares on a nonlinear model is a local search, so a bad starting point finds a bad minimum or fails to converge at all. Use physics for it: the handbook time constant, the first measurement, the obvious order of magnitude. p0 is where your prior knowledge goes.

The covariance matrix is the output almost everybody throws away, and it is the most valuable thing on this slide. The square root of its diagonal gives the standard error of each parameter, so you can quote a time constant of sixty seconds plus or minus two rather than a time constant of sixty.

Its off-diagonal entries tell you something else worth knowing: whether two parameters are trading off against each other. Strongly correlated parameters mean your data cannot separate them, and no amount of optimisation will fix that.

THE CALL, AND WHAT TO DO WITH IT

```
from scipy.optimize import curve_fit

popt, pcov = curve_fit(
    cooling_model, t, T,
    p0=[80.0, 63.7],     # from physics
)

T0_hat, tau_hat = popt
perr = np.sqrt(np.diag(pcov))

tau = 59.4 +/- 1.8 s
```

$$\sigma_i = \sqrt{C_{ii}}, \qquad \theta_i \pm 1.96\,\sigma_i$$

A parameter without an uncertainty is half a result.

THE COMPARISON

What a Fit Gives You That a Network Does Not

FOUR THINGS, AND THEY ARE NOT SMALL

Parameters that mean something. A time constant is a physical quantity with units. You can compare it to a handbook value, hand it to a colleague, or use it to design something. A network's two thousand weights mean nothing individually and cannot be compared to anything.

Uncertainty, for free, from the covariance matrix. Getting the equivalent from a neural network requires an ensemble or a Bayesian treatment, and both are considerably more work than reading a diagonal.

Extrapolation you can defend. The exponential keeps decaying outside the measured window because the physics says it does. What a network does outside its training range is a separate question with a much less comfortable answer, and lecture four point one is about that answer.

And two parameters against two thousand. With thirty-one samples, a two-parameter model is a reasonable thing to fit and a two-thousand-parameter model is a statement of faith. That ratio is the honest reason to start here.

TWO PARAMETERS AGAINST TWO THOUSAND

meaning	τ has units; w_{47} does not
uncertainty	sqrt of the covariance diagonal
extrapolation	the physics continues; a network does not
ratio	2 parameters, 31 samples

AND WHAT IT DOES NOT GIVE YOU

Flexibility. If the functional form is wrong, no amount of fitting rescues it — and that is precisely the case a network is for.

THE COMPARISON

The Baseline Deep Learning Has to Beat

THREE MODELS, ONE DATASET, AND A RANKING THAT REVERSES

Exercise three fits the same cooling curve three ways. Model A is the analytic solution with every constant from a datasheet and nothing fitted. Model B is today's curve fit — the same form, two constants estimated from the data. Model C is a neural network with eleven hundred and fifty-three parameters.

On the data they were fitted to, the ranking is what you would expect. The network has the lowest training error, by a wide margin, because it has the most freedom.

Extrapolate all three past the logged window and the ranking reverses completely. The curve fit is best by an order of magnitude. The network is wrong by tens of degrees, because a ReLU network is piecewise linear and there was nothing outside the data to put a kink in it.

The engineering consequence in that exercise is deliberately blunt. The safe handling temperature is thirty degrees; the true time to reach it is one hundred and nineteen seconds; the curve fit gets it right and the network says somewhere between seventy-five and a hundred and ten. That is tens of per cent early, on the dangerous side.

WHERE THIS CURVE COMES FROM IN POWER ELECTRONICS

The sensor on the heatsink writes the log. Model B or model C has to explain it.

THE MEASURED NUMBERS

A · analytic	1.12 °C → 0.74 °C
B · curve fit	0.32 °C → 0.05 °C
C · network	0.12 °C → tens of °C

training RMSE → extrapolation RMSE

WHY THE NETWORK LEAVES

When the Fit Fails, and How You Know

FOUR FAILURES, THREE OF WHICH ARE YOUR FAULT

A bad starting point. Nonlinear least squares is a local search, so it finds the minimum nearest to where you started. The symptom is a fit that is obviously wrong on the plot while reporting no error at all. The fix is a p0 taken from physics.

Correlated parameters. If two parameters can trade off against each other — a rate and an amplitude, often — the data cannot separate them and the covariance matrix will say so with enormous variances. The fix is more data, or fixing one parameter from independent knowledge.

The wrong functional form. This is the one that matters most and the one no diagnostic in SciPy will report. The symptom is structure in the residual plot: the fit is systematically above the data in the middle and below it at the ends. Read the residuals, not the RMSE.

And genuine non-convergence, which at least tells you. Raise maxfev, rescale your variables so they are of similar magnitude, and check the units before you assume the optimiser is at fault.

STRUCTURE IN THE RESIDUALS

A drift, not a scatter. That is a wrong constant or a wrong form — and it is invisible in the RMSE.

bad p0	wrong fit, no error reported
correlated	huge variances in pcov
wrong form	structure in the residuals
no convergence	at least it tells you

PUTTING IT TOGETHER

The Whole Thing, End to End

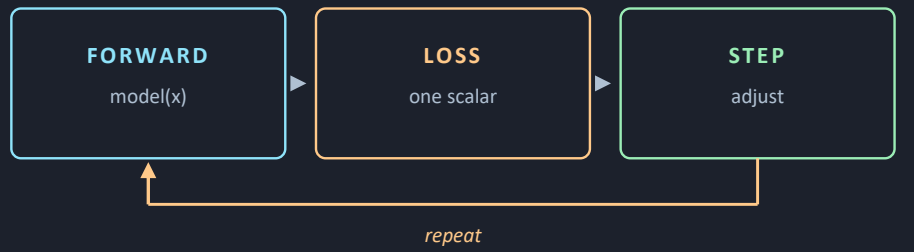
SIX STEPS, AND EVERY EXERCISE FROM HERE ON HAS THIS SHAPE

WHAT IS ACTUALLY LOAD-BEARING

Two of those six steps are where the mistakes live, and they are not the ones people expect. The cleaning step, because a file that loads is not a file that loaded correctly. And the check step, because an RMSE with no residual plot behind it can hide a completely wrong model.

The middle steps are the ones that look like the work and are the least likely to go wrong. SciPy's integrator and optimiser are mature, well tested, and rarely at fault when your answer is strange.

THE SAME SHAPE, IN L2.2



Next lecture replaces the fit with a training loop and keeps everything else. Read, clean, leave, model, fit, check — the words do not change.

THE EXERCISE

What to Do Before the Next Recording

AND AN HONEST NOTE ABOUT WHICH NOTEBOOK COVERS THIS

There is no separate exercise notebook for this half-lecture, and I would rather say so than pretend otherwise. Ex02 belongs to lecture two point two and is entirely PyTorch.

The material here is exercised in Ex03, in three weeks, where you fit the cooling curve by hand and then compare it to a network. That is the right place for it, because the comparison is the point and it needs a network to compare against.

So there are two things to do now. Finish Ex01 notebooks 02 and 03 if you have not, because lecture two point two assumes both. And spend twenty minutes typing the code blocks from today's slides into a notebook of your own against any CSV you have — a lab log, a spreadsheet export, anything.

That second one is a genuine recommendation rather than a filler task. Reading a file you did not choose is the only way to meet the six problems on slide seven, and every one of them will be in there somewhere.

WHERE THIS MATERIAL IS EXERCISED

Ex01 02–03	finish these — L2.2 assumes them
Ex02	next lecture, PyTorch, all of it
Ex03 01	the curve fit, by hand
Ex03 03	and the comparison that reverses

Ex03 fits the same block with normal equations on the log-transformed data rather than with curve_fit. Same objective, one extra subtlety about noise.

PAUSE THE VIDEO HERE

Find a CSV of your own — a lab log, an export, anything — and read it with pandas. Twenty minutes, and you will meet at least two of slide seven's six problems.

SUMMARY

Key Takeaways

1	**A file that loads may not have loaded**	Check dtypes and describe after every read. A text column where you expected floats is a file problem, found in one second or in one hour.
2	**Never fill a gap with zero**	Drop it, interpolate it, or keep the NaN — and say which you did and how many rows it affected.
3	**Every solver wants a first-order system**	A right-hand side taking t and y and returning rates. Higher order becomes higher dimension, mechanically.
4	**Check the solver against something**	An analytic case, a conserved quantity, or a tighter tolerance. L8 onwards validates networks the same three ways.
5	**Two parameters with units beat two thousand without**	A fitted physical form extrapolates, quotes an uncertainty, and can be compared to a handbook. Make deep learning earn its place.

Next — L2.2: tensors that remember what happened to them, and a differentiation engine that is not a neural-network mechanism.

SOURCES

References and Further Reading

DOCUMENTATION, AND ONE TEXTBOOK CHAPTER THAT HELPS

The pandas Getting Started guide, specifically the sections called How do I read and write tabular data and How do I handle missing values. Between them they cover slides three to ten.

The SciPy documentation for solve_ivp and for curve_fit. Both pages have a worked example at the bottom which is more useful than the argument list above it, and both are short.

Goodfellow chapter 4, on numerical computation, is the one textbook section that genuinely bears on today. It is about conditioning and about why an optimiser struggles when variables have very different magnitudes — which is the rescaling advice on slide nineteen, properly justified. It comes back in lecture six point one.

AND ONE THING NOT TO READ

Do not read the pandas API reference. It is nineteen hundred pages of method signatures and it is a reference rather than a text. Search it when you have a specific question.

IN THIS ORDER

today	pandas Getting Started, two sections
today	the solve_ivp example
before Ex03	the curve_fit example
before L6.1	GBC ch. 4 — conditioning
never	the pandas API reference, end to end

One recorded half-lecture left before the live course begins.

NEXT

L2.2 — PyTorch as an Array Library

Today you differentiated nothing and solved everything by conventional means. Next time: arrays that remember what happened to them, and a differentiation engine that will still be doing the work in week twelve. It is the most load-bearing half-lecture in Part 1.

BEFORE THE NEXT RECORDING

finish	Ex01 notebooks 02 and 03
try	pandas on a CSV of your own
read	the solve_ivp worked example
expect	Ex02, and to spend real time on it

Lecture two point two is the one to watch twice. Everything in Part 2 depends on one function call in it, and the exercise notebook that goes with it is the most important in Part 1.